

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:		Reg. No.:	
Section / Batch:		Date:	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Design and implement a Python-based resume information extraction system using Regular Expressions

Problem Understanding

A recruitment company receives resumes as plain text and needs an automated way to shortlist candidates. The system must extract structured fields — name, email, mobile number, technical skills and years of experience — from unstructured resume text, and then filter candidates who meet a minimum eligibility criterion (at least 2 years of experience and knowledge of Python).

Algorithm

- Step 1: Collect resume text as input (string per candidate).
- Step 2: Define regex patterns for each field to be extracted.
- Step 3: Apply `re.search()/re.findall()` on the resume text using each pattern.
- Step 4: Store extracted values in a dictionary (candidate profile).
- Step 5: Repeat for every resume to build a list of candidate profiles.
- Step 6: Apply the eligibility rule (experience ≥ 2 AND 'Python' in skills) to filter candidates.

Regex Pattern

Name : `Name:\s*([A-Za-z]+(?:\s[A-Za-z]+)+)`

-> matches the label 'Name:' followed by one or more capitalised words

Email : `[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}`

-> standard local-part@domain.tld structure

Mobile : `(?:\+91[-\s]?)?[6-9]\d{9}`

-> optional +91 prefix, first digit 6-9, followed by 9 more digits

Experience: `(\d+)\s+years?\s+of\s+experience`

-> captures the numeric value before the phrase 'year(s) of experience'

Skills : matched using `\b<skill>\b` against a known skill list

(Python, Java, SQL, Machine Learning, NLP), case-insensitive

Code

```
import re
resumes = [
    """
    Name: Arjun Mehta
    Email: arjun.mehta23@gmail.com
    Mobile: +91-9876543210
    Skills: Python, SQL, Machine Learning
    Experience: 3 years of experience in data analysis and NLP projects.
    """,
```

```

"""
Name: Sneha Reddy
Email: sneha_reddy@company.co.in
Phone: 9345678123
Skills: Java, SQL
Experience: 1 year of experience as a backend developer.
"""
]
NAME_PATTERN = r"Name:\s*([A-Za-z](?:\s[A-Za-z])*)"
EMAIL_PATTERN = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
MOBILE_PATTERN = r"(?:\+91[-\s])?[6-9]\d{9}"
EXPERIENCE_PATTERN = r"(\d+)\s+years?\s+of\s+experience"
SKILL_KEYWORDS = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
def extract_name(text):
    match = re.search(NAME_PATTERN, text)
    return match.group(1).strip() if match else "Not Found"
def extract_email(text):
    match = re.search(EMAIL_PATTERN, text)
    return match.group(0) if match else "Not Found"
def extract_mobile(text):
    match = re.search(MOBILE_PATTERN, text)
    return match.group(0) if match else "Not Found"
def extract_skills(text):
    found = []
    for skill in SKILL_KEYWORDS:
        # word-boundary, case-insensitive match for each known skill
        if re.search(r"\b" + re.escape(skill) + r"\b", text, re.IGNORECASE):
            found.append(skill)
    return found
def extract_experience(text):
    match = re.search(EXPERIENCE_PATTERN, text, re.IGNORECASE)
    return int(match.group(1)) if match else 0
def is_eligible(profile):
    return profile["experience"] >= 2 and "Python" in profile["skills"]
def print_summary(profile):
    print("Candidate Summary")
    print("-" * 40)
    print(f"Name      : {profile['name']}")
    print(f"Email     : {profile['email']}")
    print(f"Mobile    : {profile['mobile']}")
    print(f"Skills    : {' '.join(profile['skills']) if profile['skills'] else 'None'}")
    print(f"Experience : {profile['experience']} year(s)")
    print("-" * 40)
def main():
    print("=" * 55)
    print("RESUME INFORMATION EXTRACTION SYSTEM")
    print("=" * 55)
    all_profiles = []
    for idx, resume in enumerate(resumes, start=1):
        print(f"\nProcessing Resume #{idx} ...")

```

```

    profile = build_candidate_profile(resume)
    all_profiles.append(profile)
    print_summary(profile)
    print("\n" + "=" * 55)
    print("ELIGIBLE CANDIDATES (>= 2 years experience & Python skill)")
    print("=" * 55)
    eligible = [p for p in all_profiles if is_eligible(p)]
    if eligible:
        for p in eligible:
            print(f"- {p['name']} | {p['experience']} yrs | Skills: {' | '.join(p['skills'])}")
    else:
        print("No candidates meet the eligibility criteria.")
if __name__ == "__main__":
    main()

```

Output

```

=====
                        RESUME INFORMATION EXTRACTION SYSTEM
=====

Processing Resume #1 ...
Candidate Summary
-----
Name      : Arjun Mehta
Email     : arjun.mehta23@gmail.com
Mobile    : +91-9876543210
Skills    : Python, SQL, Machine Learning, NLP
Experience : 3 year(s)
-----

Processing Resume #2 ...
Candidate Summary
-----
Name      : Sneha Reddy
Email     : sneha_reddy@company.co.in
Mobile    : 9345678123
Skills    : Java, SQL
Experience : 1 year(s)
-----

=====
                        ELIGIBLE CANDIDATES (>= 2 years experience & Python skill)
=====

- Arjun Mehta | 3 yrs | Skills: Python, SQL, Machine Learning, NLP

```

Design and implement a Python-based product search system using Regular Expressions

Problem Understanding

An e-commerce company wants a flexible keyword-based search over its product catalogue so that customers can find products even with partial, prefixed, suffixed, or differently-cased search terms. The system needs to support five distinct search modes and also report how many products matched each type of search, which helps evaluate search effectiveness.

Algorithm

- Step 1: Maintain the product catalogue as a list of strings.
- Step 2: For each search type, construct an appropriate regex pattern.
- Step 3: Use `re.match()` for exact/prefix matches (anchored at the start) and `re.search()` for suffix/partial matches.
- Step 4: Use the `re.IGNORECASE` flag wherever case-insensitivity is required.
- Step 5: Collect all catalogue items that satisfy the pattern into a results list.
- Step 6: Display the matched product names for each search type.
- Step 7: Record the count of matches per search type into a summary report.

Regex Pattern

Exact match : `^<keyword>$` -> string equals keyword exactly

Prefix search : `^<prefix>` -> string starts with prefix

Suffix search : `<suffix>$` -> string ends with suffix

Partial search: `<substring>` (`re.search`) -> substring occurs anywhere

Case-insens. : same patterns + `re.IGNORECASE` flag

Code

```
import re
products = [
    "Wireless Mouse",
    "Wireless Keyboard",
    "Bluetooth Speaker",
    "Bluetooth Headphones",
    "Gaming Mouse",
    "Gaming Chair",
    "USB Cable",
    "USB-C Charger",
    "Smart Watch",
    "Smartphone Stand",
```

```

    "wireless earbuds",
]
def exact_search(keyword, catalogue):
    pattern = r"^" + re.escape(keyword) + r"$"
    return [p for p in catalogue if re.match(pattern, p)]
def prefix_search(prefix, catalogue):
    pattern = r"^" + re.escape(prefix)
    return [p for p in catalogue if re.match(pattern, p, re.IGNORECASE)]
def suffix_search(suffix, catalogue):
    pattern = re.escape(suffix) + r"$"
    return [p for p in catalogue if re.search(pattern, p, re.IGNORECASE)]
def partial_search(substring, catalogue):
    pattern = re.escape(substring)
    return [p for p in catalogue if re.search(pattern, p, re.IGNORECASE)]
def case_insensitive_search(keyword, catalogue):
    pattern = r"^" + re.escape(keyword) + r"$"
    return [p for p in catalogue if re.match(pattern, p, re.IGNORECASE)]
def display_results(title, results):
    print(f"\n{title}")
    print("-" * 45)
    if results:
        for r in results:
            print(f"  -> {r}")
    else:
        print(" No matching products found.")
    print(f"Total Matches: {len(results)}")
def main():
    print("=" * 50)
    print("PRODUCT SEARCH SYSTEM")
    print("=" * 50)
    report = {}
    q = "Bluetooth Speaker"
    res = exact_search(q, products)
    display_results(f'Exact Search for '{q}', res)
    report["Exact Search"] = len(res)
    q = "Wireless"
    res = prefix_search(q, products)
    display_results(f'Prefix Search for '{q}', res)
    report["Prefix Search"] = len(res)
    q = "Mouse"
    res = suffix_search(q, products)
    display_results(f'Suffix Search for '{q}', res)
    report["Suffix Search"] = len(res)
    q = "phone"
    res = partial_search(q, products)
    display_results(f'Partial Search for '{q}', res)
    report["Partial Search"] = len(res)
    q = "wireless keyboard"
    res = case_insensitive_search(q, products)
    display_results(f'Case-Insensitive Search for '{q}', res)

```



```

report["Case-Insensitive Search"] = len(res)
print("\n" + "=" * 50)
print("SEARCH SUMMARY REPORT")
print("=" * 50)
for search_type, count in report.items():
    print(f"{search_type:<25}: {count} product(s) found")
if __name__ == "__main__":
    main()

```

Output

PRODUCT SEARCH SYSTEM

Exact Search for 'Bluetooth Speaker'

-> Bluetooth Speaker

Total Matches: 1

Prefix Search for 'Wireless'

-> Wireless Mouse
-> Wireless Keyboard
-> wireless earbuds

Total Matches: 3

Suffix Search for 'Mouse'

-> Wireless Mouse
-> Gaming Mouse

Total Matches: 2

SEARCH SUMMARY REPORT

Exact Search	: 1 product(s) found
Prefix Search	: 3 product(s) found
Suffix Search	: 2 product(s) found
Partial Search	: 2 product(s) found
Case-Insensitive Search	: 1 product(s) found

Design and implement a Python-based validation system using Regular Expressions

Problem Understanding

A university's online registration portal must validate student-entered data before allowing course enrolment. Five fields need validation: register number, institutional email, course code, semester, and mobile number. Each field follows a specific institutional format, so regex is used to check the format of the input and generate a validation message per field, followed by an overall registration status.

Algorithm

- Step 1: Define the expected format for each field as a regex pattern.
- Step 2: Read/accept student input values for all five fields.
- Step 3: Validate each field independently using `re.match()` against its pattern.
- Step 4: Store True/False validity result per field in a results dictionary.
- Step 5: Display a VALID/INVALID message for every field.
- Step 6: Compute overall status: registration succeeds only if ALL fields are valid.
- Step 7: Print the final registration status report for every student.

Regex Pattern

Register No : `^\d{2}[A-Z]{3}\d{3}$`

-> 2-digit year + 3 uppercase letters (dept) + 3 digits, e.g. 22CSE101

Inst. Email : `^[a-zA-Z0-9._%+-]+@university\.edu\.in$`

-> must end with the official university domain

Course Code : `^[A-Z]{2,4}\d{2,3}$`

-> 2-4 uppercase letters followed by 2-3 digits, e.g. NLP03

Semester : `^[1-8]$`

-> a single digit between 1 and 8

Mobile No : `^(?:\+91[-\s])?[6-9]\d{9}$`

-> optional +91 prefix, first digit 6-9, 10 digits total

Code

```
import re
REGISTER_NO_PATTERN = r"^\d{2}[A-Z]{3}\d{3}$"
EMAIL_PATTERN = r"^[a-zA-Z0-9._%+-]+@university\.edu\.in$"
COURSE_CODE_PATTERN = r"^[A-Z]{2,4}\d{2,3}$"
SEMESTER_PATTERN = r"^[1-8]$"

```

```

MOBILE_PATTERN = r"^(?:\+91[-\s]?)?[6-9]\d{9}$"
def validate_field(pattern, value):
    return bool(re.match(pattern, value.strip()))
def validate_student(student):
    results = {}
    results["Register Number"] = validate_field(REGISTER_NO_PATTERN,
student["register_no"])
    results["Institutional Email"] = validate_field(EMAIL_PATTERN,
student["email"])
    results["Course Code"] = validate_field(COURSE_CODE_PATTERN,
student["course_code"])
    results["Semester"] = validate_field(SEMESTER_PATTERN, student["semester"])
    results["Mobile Number"] = validate_field(MOBILE_PATTERN,
student["mobile"])
    return results
def print_validation_report(name, values, results):
    print(f"\nValidation Report for: {name}")
    print("-" * 55)
    for field, is_valid in results.items():
        status = "VALID" if is_valid else "INVALID"
        print(f'{field:<22}: {values[field]:<20} -> {status}')
    print("-" * 55)
    overall = all(results.values())
    final_status = "REGISTRATION SUCCESSFUL" if overall else
"REGISTRATION FAILED"
    print(f'Final Status: {final_status}')
    return overall
def main():
    print("=" * 55)
    print("UNIVERSITY ONLINE REGISTRATION VALIDATION SYSTEM")
    print("=" * 55)
    students = [
        {
            "name": "Divya Prakash",
            "register_no": "22CSE101",
            "email": "divya.prakash@university.edu.in",
            "course_code": "NLP03",
            "semester": "6",
            "mobile": "+91-9876543210",
        },
        {
            "name": "Rohit Sharma",
            "register_no": "22CS101",
            "email": "rohit.sharma@gmail.com",
            "course_code": "cs101",
            "semester": "9",
            "mobile": "12345",
        },
    ]
    summary = []

```

```

for student in students:
    values = {
        "Register Number": student["register_no"],
        "Institutional Email": student["email"],
        "Course Code": student["course_code"],
        "Semester": student["semester"],
        "Mobile Number": student["mobile"],
    }
    results = validate_student(student)
    overall = print_validation_report(student["name"], values, results)
    summary.append((student["name"], overall))
print("\n" + "=" * 55)
print("FINAL REGISTRATION STATUS REPORT")
print("=" * 55)
for name, status in summary:
    print(f'{name:<20}: {'SUCCESSFUL' if status else 'FAILED'}')
if __name__ == "__main__":
    main()

```

Output

UNIVERSITY ONLINE REGISTRATION VALIDATION SYSTEM

Validation Report for: Divya Prakash

```

-----
Register Number      : 22CSE101                -> VALID
Institutional Email  : divya.prakash@university.edu.in -> VALID
Course Code          : NLP03                    -> VALID
Semester             : 6                        -> VALID
Mobile Number        : +91-9876543210          -> VALID
-----

```

Final Status: REGISTRATION SUCCESSFUL

Validation Report for: Rohit Sharma

```

-----
Register Number      : 22CS101                -> INVALID
Institutional Email  : rohit.sharma@gmail.com -> INVALID
Course Code          : cs101                    -> INVALID
Semester             : 9                        -> INVALID
Mobile Number        : 12345                    -> INVALID
-----

```

Final Status: REGISTRATION FAILED

FINAL REGISTRATION STATUS REPORT

Divya Prakash : SUCCESSFUL

Rohit Sharma : FAILED

Overall Conclusion

All three systems demonstrate the core strength of Regular Expressions in NLP-related text processing: the ability to detect, extract, and validate structured information from unstructured or semi-structured text using compact, rule-based patterns. Regex-based information extraction (Q1), flexible pattern matching (Q2), and input validation (Q3) together cover the three major application areas of regex highlighted in the CO1 statement of this course.